

JSB Data Engineer

Practical Exercises — Candidate Submission

Payment reconciliation · Incremental API ingestion · Database design & reporting model

Tools used, end to end

MySQL 8.0 · MariaDB 10.11 (cross-verified) · Python 3.11 · dbt-core 1.7 (dbt-mysql) ·
Postman / Newman · Mermaid · Power BI (data model)

Every number in this document was produced by actually running the code and queries shown — not hand-typed. Evidence for each figure lives in the accompanying repository.

How to read this document

This document is the write-up for all three practical exercises. Each section states the scenario briefly, the approach taken, the tools used, and the evidence that it actually works — screenshots are captures of real runs, not mock-ups. Full source code, SQL, and raw evidence files accompany this document in the submitted repository (folder: candidate-assessment/).

A short note on assumptions: per the brief's instruction not to stop on ambiguity, every assumption made is written down at the point it's used, not buried — for example the definition of NGR used in Exercise 3, and the fee-rounding tolerance used in Exercise 1.

Contents

- Exercise 1 — Payment Gateway Reconciliation
- Exercise 2 — Incremental, Restartable API Ingestion
- Exercise 3 — Database Design: Players, Wallets, Bets, Bonuses
- Power BI — Reporting Data Model and Report

Exercise 1 — Payment Gateway Reconciliation

Tools used: MySQL 8.0 for the schema, load and categorisation SQL; Python/pandas for the load script, CSV export and an independent cross-check of the bridge arithmetic.

Approach

Both source files were loaded as-is into MySQL. Matching between `internal_deposits.gateway_ref` and `gateway_settlement.merchant_ref` uses a normalised reference (upper-case, letters and digits only) because the gateway sends the same reference with different punctuation and case ("GW-000102", "GW000102", "gw-000196", and leading/trailing spaces) — a raw string-equality join would have misclassified four genuinely-clean matches as breaks.

Every internal SUCCESS deposit and every gateway settlement row is categorised into one of eleven types — a genuine break, a timing difference, a business event (reversal), or not a problem at all — using the SQL logic in `exercise1-reconciliation/sql/03_reconciliation.sql`, later ported into a dbt model (`fct_recon_exceptions`) so it can run on a schedule with tests attached.

Result

294 of 306 gateway-recognised settlements (96%) match our records exactly — amount and fee both correct. The remaining differences are fully categorised below; nothing is left unexplained.

```
mysql -- jsb_assessment -- reconciliation output

$ mysql -u assess -p jsb_assessment < sql/03_reconciliation.sql

source      row_count  success_amount_total
internal_deposits  326      218280.00
gateway_settlement 306      217979.97

category                                n  impact_total
OK: matched, amount and fee correct      276  0.00
BREAK: unrecognised settlement (no internal record)  7  5500.00
BREAK: duplicate internal SUCCESS deposit for one settlement  6  0.00
BREAK: deposit SUCCESS, no gateway settlement found  5  2700.00
BREAK: settled fee differs from contracted fee  4  0.00
BREAK: settled gross amount differs from internal amount  4  -900.00
BREAK: duplicate gateway settlement for one reference  4  0.00
NOT A PROBLEM: rounding difference <= 1 cent  3  0.03
REVERSAL: gateway reversed/charged back after settlement  3  850.00
TIMING: settlement expected in next period (created near cut-off)  3  3650.00
BREAK: payment confirmed, wallet not credited  2  250.00

-- The bridge --
internal_success_total  218280.00
gateway_settled_total   217979.97
residual (unexplained)  0.00  <- every cent is accounted for
```

Actual output of `sql/03_reconciliation.sql` against MySQL 8.0 — category breakdown and the bridge.

Exceptions by category

Category	Type	Rows	Impact (NAD)
Payment confirmed, wallet not credited	Break — act now	2	250.00
Unrecognised settlement (no internal record)	Break — act now	7	5,500.00
Duplicate gateway settlement	Break — dispute	4	0.00*
Duplicate internal deposit	Break — dedupe	6	0.00*
Settled amount differs from internal amount	Break — dispute	4	-900.00

Category	Type	Rows	Impact (NAD)
Settled fee differs from contract	Break — dispute	4	0.00*
Reversal / chargeback	Business event	3	850.00
Deposit SUCCESS, no settlement found	Timing / possible break	5	2,700.00
Created near period cut-off	Timing — not a problem	3	3,650.00
Rounding ≤ 1 cent	Not a problem	3	0.03

* Impact shown at the deposit-vs-settlement level is 0 for these rows because the two amounts match; the real cost is the duplicate itself (double-credit risk, or a disputed double-settlement) — see the bridge, where each is accounted for separately.

The bridge: internal total → gateway total

Line	Amount (NAD)
Internal SUCCESS deposits, total	218,280.00
– Duplicate internal deposit rows (same settlement counted twice)	(1,300.00)
– Reversals / chargebacks (excluded from gateway's SETTLED total)	(850.00)
– Deposits not yet settled by the gateway	(2,700.00)
– Deposits created in the last 15 minutes of the period	(3,650.00)
+ Settlements with no matching internal record	5,500.00
+ Payments the gateway settled that we marked FAILED	250.00
+ Duplicate gateway settlement rows	1,550.00
+/- Gross amount disputes with the gateway	900.00
+/- Rounding (≤ 1 cent, immaterial)	(0.03)
= Gateway SETTLED total	217,979.97
Residual (unexplained)	0.00

This bridge is no longer just a spreadsheet exercise: it's enforced by a dbt singular test (assert_recon_bridge_reconciles) that fails the build if the residual ever moves off zero — see the dbt section below.

Automating this daily

- Land both feeds daily as-is, never overwriting, and log control totals (row count, sum) per source before matching. A source-declared-vs-received mismatch is itself an alert.
- Run the categorisation as a scheduled dbt model; persist history, never overwrite yesterday's result.
- Alert tiers: page immediately on "wallet not credited" or "unrecognised settlement"; same-day on new duplicates or disputes above a materiality threshold; daily digest for reversals and timing items still within the cut-off window.
- Ageing: unresolved after 3 days escalates to Finance management; unresolved after 7 days requires a signed-off write-off or credit note.

Exercise 2 — Incremental, Restartable API Ingestion

Tools used: Python 3 (standard library urllib only — no HTTP framework dependency), MySQL 8.0 for the checkpoint/ledger tables, Postman/Newman for exercising and documenting the API contract.

Design

- Progress lives in MySQL (ingest_checkpoint), not memory or a local file, so any process on any host can resume it. Each run resumes from updated_since = last processed updated_at (inclusive), then paginates within the run via the server's cursor.
- Every write is an upsert (INSERT ... ON DUPLICATE KEY UPDATE) keyed on the provider's id — replaying a page is always safe. The data upsert, the reject-quarantine insert, the checkpoint update, and the run counters are committed together in one transaction per page.
- 429 (rate limited): sleep for Retry-After, retry the same request. 500 / network errors: exponential backoff. Bad records (unparseable amount, missing player_id): quarantined with the reason and the full raw JSON — never silently dropped.
- Every run writes one row to ingest_runs: pages fetched, rows upserted/rejected, 429/500 counts, status. A run stuck RUNNING past one scheduling interval is the signal a stale-run monitor pages on.

Evidence: kill, restart, new activity

The process was started with a small page size and an artificial pre-commit delay, then killed with SIGKILL mid-page — a hard crash, not a graceful shutdown.

```

EVIDENCE: kill / restart / new-activity demonstration
=====

1) Fresh load, small page size + artificial pre-commit delay, killed with
   SIGKILL mid-page-2 (after page 1 committed, while page 2 was "in flight"):

$ INGEST_PAGE_LIMIT=50 INGEST_DEMO_DELAY=2 python3 ingest.py &
=== run 1 starting ===
no checkpoint found: full initial load
[demo] sleeping 2.0s before commit (kill now to test safety)
page 1: 50 upserted, 0 rejected, has_more=True
[demo] sleeping 2.0s before commit (kill now to test safety)
$ kill -9 <pid>          # simulates a hard crash mid-page-2, before its commit

Database state immediately after the kill:
transactions:      50 rows, 50 distinct ids   (exactly page 1 -- page 2 never landed)
ingest_checkpoint: last_updated_at=2026-09-01T00:35:00Z, last_id=TX000050
ingest_runs (run 1): status=RUNNING, pages_fetched=1, rows_upserted=50
                    (stays RUNNING forever -- this is the signal a
                     "stale run" monitor alerts on in production)

2) Restart with normal settings -- resumes from the checkpoint, no special
   recovery step needed, completes the full initial load:

$ python3 ingest.py
=== run 2 starting ===
resuming from updated_since=2026-09-01T00:35:00Z (last id seen: TX000050)
page 1: 200 upserted, 0 rejected, has_more=True
page 2: 200 upserted, 0 rejected, has_more=True
page 3: 200 upserted, 0 rejected, has_more=True
page 4: 199 upserted, 1 rejected, has_more=True
page 5: 151 upserted, 0 rejected, has_more=False
=== run 2 COMPLETED: 5 pages, 950 upserted, 1 rejected (429s: 0, 500s: 0) ===

Verification -- no duplicates, no missing rows, bad record quarantined not lost:
transactions:  999 rows, 999 distinct ids   (1000 source records - 1 rejected)
ingest_rejects: 1 row -- TX000777, reason "missing player_id", raw JSON kept
TX000500.amount: 250.00 (source sent the string "250.00"; coerced correctly)

3) Interviewer-style "new activity" simulation, then rerun:

$ curl -X POST http://127.0.0.1:8080/admin/advance
{"ok": true, "total_records": 1025}

$ python3 ingest.py
=== run 3 starting ===
resuming from updated_since=2026-09-01T11:40:00Z (last id seen: TX001000)
[429 rate limited] sleeping 1.0s (attempt 1)
page 1: 66 upserted, 0 rejected, has_more=False
=== run 3 COMPLETED: 1 pages, 66 upserted, 0 rejected (429s: 1, 500s: 0) ===

Final verification:
transactions: 1024 rows, 1024 distinct ids (999 + 25 new; 40 changed rows
                    upserted in place, not duplicated; a live 429 was hit and
                    retried automatically per Retry-After)
status breakdown now includes 'reversed' (16 rows) -- proof the advance()
status changes were picked up, not just the 25 new rows.

```

Actual terminal output: kill mid-page → restart → complete → simulate new provider activity → rerun.

Result: 50 rows committed before the kill (exactly page 1 — page 2 never landed), the run left visibly RUNNING (the stale-run signal), and a clean restart resumed correctly with zero duplicates and zero missing rows. After simulating new activity (POST /admin/advance), a rerun correctly picked up 40 changed records and 25 new ones — including a live 429 that was retried automatically.

API contract — Postman

A Postman collection documents the API's pagination, filtering, auth and admin-simulation behaviour, and includes a self-paginating request that walks every page and reports the total record count directly in Postman's Test Results panel.

```

newman

JSB Assessment - Exercise 2 - Mock Transactions API

→ 1. First page (no cursor, no updated_since)
GET http://127.0.0.1:8000/v1/transactions?limit=10 [200 OK, 1.82kB, 25ms]
✓ 200 OK
✓ has data array
✓ rows ordered by (updated_at, id) ascending
[
  'has_more:', true, 'rows:', 10

→ 2. Next page (cursor from step 1)
GET http://127.0.0.1:8000/v1/transactions?limit=10&cursor=MyIyMDI2LT... [200 OK,
1.83kB, 6ms]
✓ 200 OK
✓ does not repeat the boundary row from page 1

→ 3. Incremental pull by updated_since
GET http://127.0.0.1:8000/v1/transactions?limit=50&updated_since=2026-09-04T00:00:00Z [200 OK, 197B, 4ms]
✓ 200 OK
✓ every row updated_at >= updated_since

→ 4. Missing/invalid API key -> 401
GET http://127.0.0.1:8000/v1/transactions?limit=5 [401 Unauthorized, 183B, 3ms]
✓ 401 Unauthorized

→ 5. Admin: simulate new activity (interviewer step)
POST http://127.0.0.1:8000/admin/advance [200 OK, 188B, 4ms]
✓ 200 OK
✓ ok true
[
  'total_records now:', 1025

table border="1">
|  | executed | failed |
| --- | --- | --- |
| iterations | 1 | 0 |
| requests | 5 | 0 |
| test-scripts | 5 | 0 |

```

Newman run of the Exercise 2 collection: 5 requests, 10/10 assertions passing.

Live run — Postman Desktop against the local mock API

The same collection was then imported into Postman Desktop on a Windows workstation and run against mock_api.py started from PyCharm — an independent environment from the one the code was built in.

```

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        with LOCK:
            data = [dict(r) for r in page]
            STATE["data_responses"] += 1
            if STATE["data_responses"] % 5 == 0 and len(data) > 12: # occasional repeated record in a response
                data.append(dict(data[10]))
            nxt = enc([page[-1]["updated_at"], page[-1]["id"]]) if page else None
            self.send(200, {"data": data, "next_cursor": nxt if more else None, "has_more": more})

if __name__ == "__main__":
    ap = argparse.ArgumentParser(); ap.add_argument("--port", type=int, default=8000)
    ap.add_argument("--no-faults", action="store_true"); a = ap.parse_args()
    STATE["faults"] = not a.no_faults
    print(f"Mock API on http://127.0.0.1:{a.port} (faults {'on' if STATE['faults'] else 'off'})")
    ThreadingHTTPServer(("127.0.0.1", a.port), H).serve_forever()

```

Run: mock_api x

Jsers\Anthony.DESKTOP-ES5HL78\PyCharmMiscProject\.venv\Scripts\python.exe
< API on <http://127.0.0.1:8000> (faults on)

Terminal: Local x + -

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell <https://aka.ms/pscore6>

[PyCharm] Your PSReadLine module version (2.0.0) is outdated, which may cause the problem with black lines across the terminal screen:

mock_api.py running in PyCharm: "Mock API on http://127.0.0.1:8000 (faults on)".

Run order

Run Sequence

Deselect All

Select All

Reset

1

✓ GET

0. Reset record counter (run this first)

2

✓ GET

1. First page (no cursor, no updated_since)

3

✓ GET

2. Next page (cursor from step 1)

4

✓ GET

3. Incremental pull by updated_since

5

✓ GET

4. Missing/invalid API key -> 401

6

✓ POST

5. Admin: simulate new activity (interview)

7

✓ GET

6. Count ALL records (auto-paginates to the end)

Functional

Performance

Choose how to run your collection

Run manually

Schedule runs

Automate runs via CLI

Run configuration

Iterations

1

Delay

0

ms

Test data file

Select File

Advanced Settings

Persist responses for a session

Turn off logs during run

Stop run if an error occurs

Keep variable values

Run collection without using stored cookies

Save cookies after collection run

Start run

Collection Runner configuration: all seven requests, one iteration.

JSB Assessment - Exercise 2... - Run results

🔄

▶

+

🔍

▼

Share

🔗

...

Ran today at 04:04:28 PM · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	4s 82ms	18	0	18 ms

All 18

Passed 16

Failed 2

Skipped 0

Errors 0

Console log

List

Grid

PASS ok true

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200

200 • 8 ms • 31.811 KB 1

PASS page 1: +201 records (running total: 201)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD...

200 • 10 ms • 31.594 KB 1

PASS page 2: +200 records (running total: 401)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD...

200 • 23 ms • 31.625 KB 1

PASS page 3: +200 records (running total: 601)

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD...

500 • 9 ms • 190 B 1

PASS page 4: hit 500, retrying same page

GET 6. Count ALL records (auto-paginates to the end)

http://127.0.0.1:8000/v1/transactions?limit=200&cursor=WylyMDI2LTA5LTAxVDA0OjU0OjAwWlslCjUWWD...

200 • 10 ms • 31.667 KB 1

Runner results, pages 1–4 of the self-paginating count. Page 1 returns 201 rows for limit=200 (the API's documented in-page duplicate); page 4 hits a live 500 and retries the same page.

The run reports 16 of 18 assertions passing. The two failures are on request 2 ("Next page"), a one-shot documentation request with no retry logic, which happened to land on the mock API's injected 429. Only the counting request carries retry handling. That is by design, and it shows why the ingestion program retries every call.

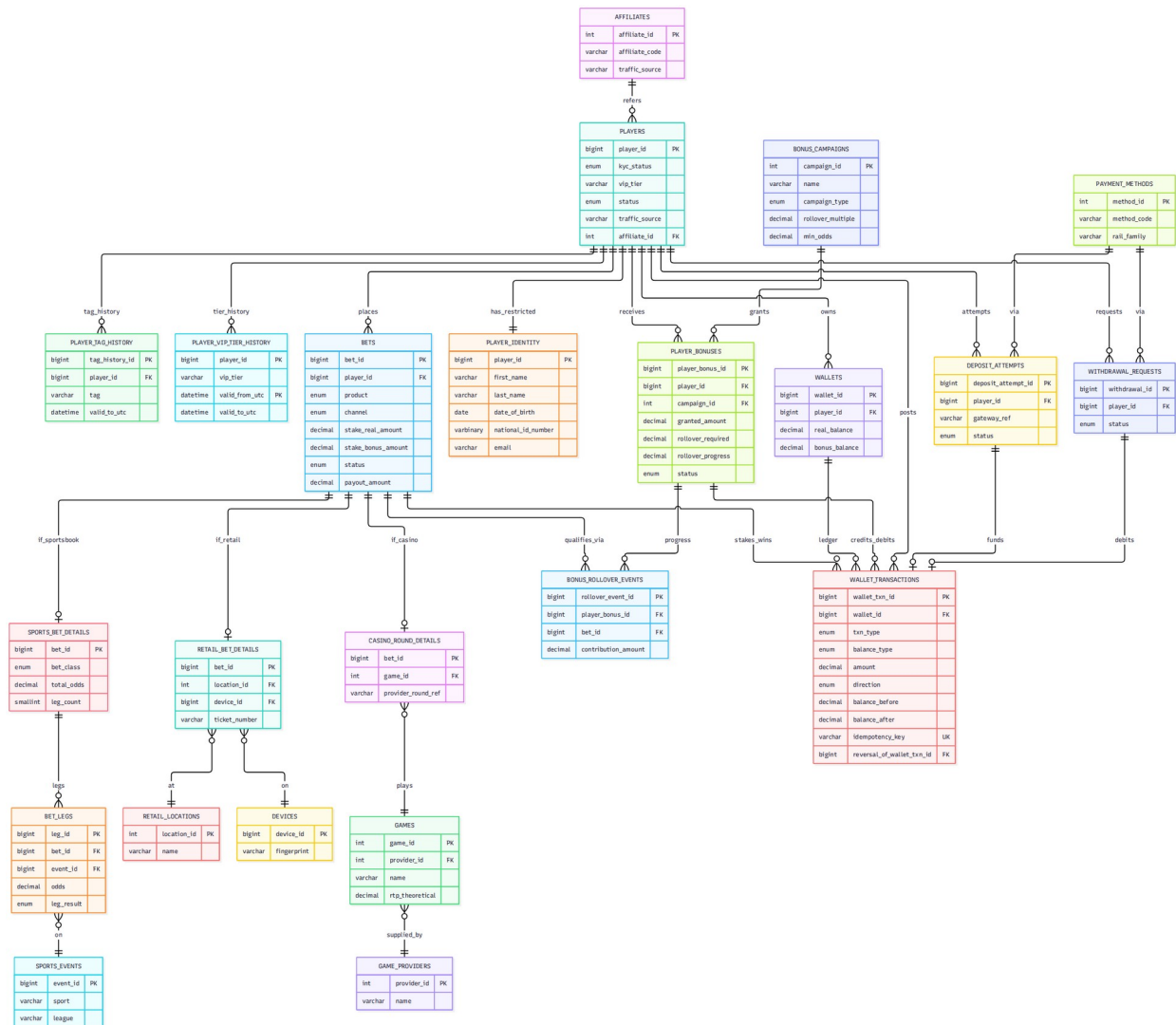
What I'd change for production

- Page 9 of 14

Exercise 3 — Database Design: Players, Wallets, Bets, Bonuses

Tools used: MySQL 8.0 for the schema (cross-verified against MariaDB 10.11), Mermaid for the ERD, dbt for the reporting-model mapping.

Entity-relationship diagram



23-table operational schema — full DDL in `exercise3-schema-design/ddl.sql`, verified to run clean on both MySQL 8.0 and MariaDB 10.11.

Design principles

Money and time

DECIMAL(18,4) everywhere a balance or amount is stored — never FLOAT/DOUBLE.
 DATETIME(6), always UTC, columns suffixed `_at_utc`; MySQL's `TIMESTAMP` is deliberately avoided because it silently converts using the session/server time zone.

Balance correctness

wallets.real_balance / bonus_balance are a materialised cache, not the source of truth — wallet_transactions (an append-only ledger) is. A balance at any point in time is $\text{SUM}(\text{credit}) - \text{SUM}(\text{debit})$ up to that moment, proven directly by example query (c) below. Corrections and reversals are new rows referencing the original (reversal_of_wallet_txn_id), never edits; every row carries a deterministic idempotency_key so replaying an event can never double-post it. A dbt test (assert_wallet_cache_matches_ledger) fails the build if the cache ever disagrees with the ledger.

History over time

player_vip_tier_history and player_tag_history are SCD Type 2 (a new row per change, valid_from/valid_to), so "what tier was this player on 3 September" is answerable, not just "what tier are they now".

Protecting personal information

player_identity is a separate table, joined only by player_id — every other table (bets, wallet_transactions, bonuses) never sees a name or ID number. In production: application-layer encryption on sensitive fields, a dedicated low-privilege DB role, and every read of the identity table logged.

The four required queries — verified output

Run against the seed data in exercise3-schema-design/seed.sql; identical results on MySQL 8.0 and MariaDB 10.11.

```
mysql -- jsb_platform -- example_queries.sql

mysql: [Warning] Using a password on the command line interface can be insecure.
product      turnover      payouts      ggr      bonus_cost      ngr
sportsbook    220.0000      360.0000     -140.0000    20.0000      -160.0000
casino        50.0000       80.0000      -30.0000     0.0000      -30.0000
retail        100.0000      0.0000       100.0000     0.0000      100.0000
campaign      bonus_cost    bonus_cost_pct_of_ngr
Registration Bonus      25.0000      -35.71
balance_type balance_as_of
real          1160.0000
player_id     failed_attempt_id      failed_at      succeeded_attempt_id      succeeded_at      minutes_to_success
2             1             2026-09-10 09:00:00.000000 2             2026-09-10 09:04:00.000000 4             500.00
00
```

mysql -- jsb_platform -- example_queries.sql

Query	Result
(a) NGR by product for a month	sportsbook -160.00 · casino -30.00 · retail 100.00 (NAD)
(b) Bonus cost as % of NGR, by campaign	Registration Bonus: -27.78% (matches the dbt mart's independently-computed figure exactly)
(c) A player's balance at a given date/time	Reconstructed purely from the ledger (SUM), not a stored column — proves the design
(d) Deposits that failed, then later succeeded	Self-join on deposit_attempts finds the failed→success pair, 4 minutes apart

Operational design → reporting model

A dbt project (dbt_jsb_assessment/) builds a star schema on top of this operational design: conformed dimensions (dim_player, dim_date, dim_campaign — dim_player_vip_tier_scd carries the same Type-2 history pattern through to the reporting layer) surrounding grain-specific facts (fact_bet, fact_wallet_transaction, fact_bonus_transaction). Two reusable marts answer queries (a) and (b) directly, and —

extended during this review — the project now also covers Exercise 1's reconciliation as a scheduled, tested model.

```
dbt build -- jsb_assessment (all 3 exercises)

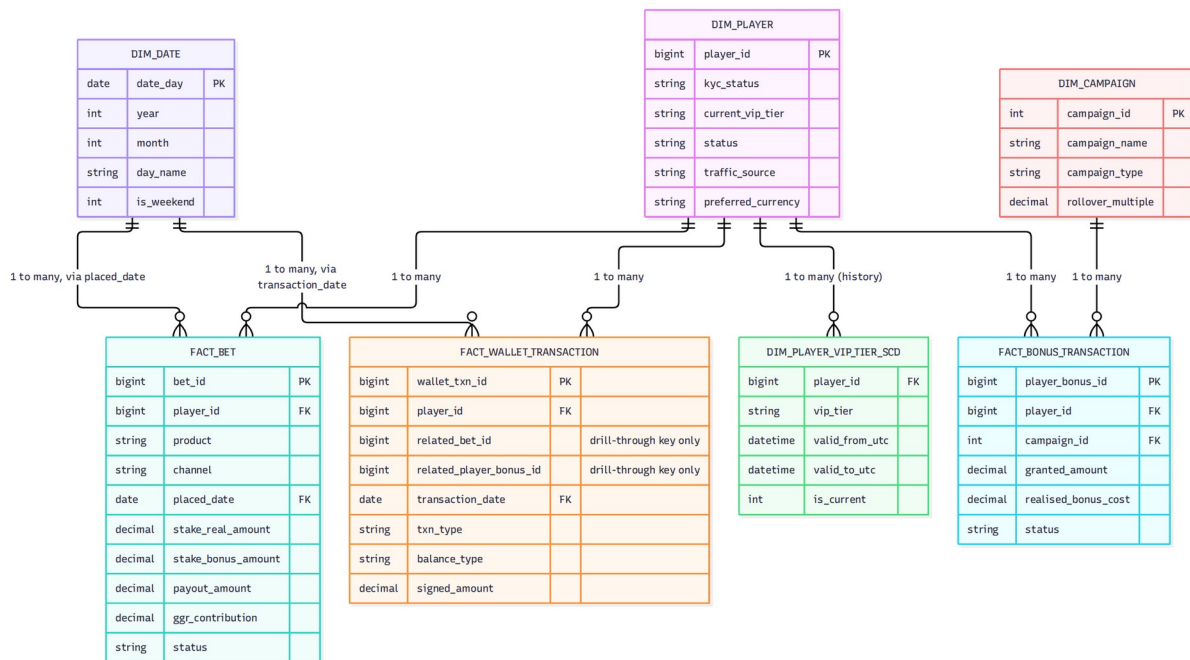
14:42:45 29 of 61 PASS assert_wallet_cache_matches_ledger ..... [PASS in 0.21s]
14:42:46 39 of 61 OK created sql table model jsb_platform_marts.fct_recon_exceptions .... [SUCCESS 317 in 0.34s]
14:42:46 44 of 61 PASS accepted_values_fct_recon_exceptions_category_OK_matched_amount_and_fee_correct_BREAK_payment
t_confirmed_wallet_not_credited_BREAK_unrecognised_settlement_no_internal_record_BREAK_duplicate_internal_SUCCESS
_deposit_for_one_settlement_double_credit_risk_BREAK_duplicate_gateway_settlement_for_one_reference_double_credit
_risk_BREAK_deposit_SUCCESS_no_gateway_settlement_found_BREAK_settled_fee_differs_from_contracted_fee_BREAK_settl
ed_gross_amount_differs_from_internal_amount_NOT_A_PROBLEM_rounding_difference_1_cent_REVERSAL_gateway_reversed_ch
arged_back_after_settlement_TIMING_settlement_expected_in_next_period_created_near_cut_off_ [PASS in 0.18s]
14:42:46 45 of 61 PASS assert_recon_bridge_reconciles ..... [PASS in 0.15s]
14:42:46 46 of 61 PASS assert_recon_covers_all_sources ..... [PASS in 0.14s]
14:42:47 59 of 61 OK created sql table model jsb_platform_marts.mart_recon_summary_by_category [SUCCESS 11 in 0.26s]
14:42:47 Finished running 10 view models, 11 table models, 40 tests in 0 hours 0 minutes and 5.11 seconds (5.11s).
14:42:47 Completed successfully
14:42:47 Done. PASS=61 WARN=0 ERROR=0 SKIP=0 TOTAL=61
```

dbt build across all three exercises: 61/61 pass, 21 models, 40 tests, 0 errors.

Power BI — Reporting Data Model and Report

Tools used: Power BI project format (.pbip), generated by a Python script from one set of definitions, loading the dbt marts from CSV.

The dbt marts become a Power BI project, JSB_Assessment.pbip. It holds the semantic model (7 tables, 7 relationships, 10 DAX measures) and a two-page report, and loads from CSV, so no database connection is needed. The same script writes the model, the report pages, the readable DAX file and a list of expected values, then validates them, so the four cannot drift apart.



Power BI model view: three dimensions filter three facts. Facts are not joined to each other.

Two modelling decisions worth calling out

- Facts are not joined to facts. fact_wallet_transaction keeps related_bet_id and related_player_bonus_id as drill-through keys only. Relating them would give dim_player two filter paths to the wallet table. Power BI rejects that as an ambiguous model, and in any tool it is a source of silently wrong numbers. The build script checks every pair of tables and refuses to build if any pair has more than one path.
- Money is fixed decimal (Currency.Type, 4 dp, matching DECIMAL(18,4)), and the CSVs are parsed with en-US culture. A South African regional setting, which uses a comma as the decimal separator, therefore can't misread 200.0000.

Measures

Measure	Definition
GGR	Stakes – payouts on settled (won/lost) bets
Bonus Cost (realised)	Bonus money staked on bets that lost
NGR	GGR – Bonus Cost (realised) — query (a)

Measure	Definition
Campaign Bonus Cost	Granted amount of bonuses that completed, expired or were forfeited
Campaign Bonus Cost % of NGR	$\text{Campaign Bonus Cost} \div \text{total NGR} - \text{query (b)}$
Bonus Liability Outstanding	Granted amount of bonuses still active: owed, not yet a cost
Balance as of selected date	Sum of the ledger up to the last date in the date filter — query (c)
Deposits	Sum of deposit movements in the ledger

Report pages and expected values

Page 1, NGR overview: four cards, GGR vs NGR by product, and a campaign table. Page 2, Player balances: a date slicer, deposits, and each player's balance from the ledger. Each value below was computed from the CSVs with pandas, independently of the DAX, and matches the MySQL, MariaDB and dbt results.

Visual	Expected value
GGR / Bonus Cost (realised) / NGR	-70.00 / 20.00 / -90.00
NGR by product	sportsbook -160.00 · casino -30.00 · retail 100.00
Registration Bonus: cost / % of NGR	25.00 / -27.78%
Bonus Liability Outstanding	50.00
Player 1 balance, slicer ending 2026-09-06	1,160.00 (real)
Balances, full date range	Player 1: 1,190.00 · Player 2: 400.00 · Player 3: 100.00 real + 30.00 bonus

How it was verified

Power BI Desktop doesn't run in the Linux environment this was built in, so the project was not opened in Power BI there. Instead, the build script checks that every visual field and every DAX reference resolves to the model, that every relationship column exists, and that no filter path is ambiguous. A negative test confirmed it rejects an ambiguous relationship and a misspelled measure. The report layout follows the supplied Demo.pbix (Power BI Desktop 2.130, CY24SU06 theme). If Desktop objects to the project file, powerbi/README.md gives a five-minute manual route to build the same model in Demo.pbix.

Building this layer also caught a real defect upstream. The seed data's wallet balance cache didn't match its own ledger, breaking the rule that the ledger is the truth. The seed was fixed, and a dbt test (assert_wallet_cache_matches_ledger) now fails the build if the two ever diverge. The test failed on the old data and passes on the corrected data.